

検索アルゴリズム

1 はじめに

与えられた集合 S の中に x が要素として含まれているかどうかを調べる探索問題について紹介します。探索問題はさまざまなさまざまな場面で現れる典型的な問題です。例えば与えられた社員リストの中から特定の社員がいるかどうかを探すなどさまざまな状況が考えられます。そのような時には、データの数が少ないければ順番に探索していけば良いでしょうが、データの数が多くなると効率の良いアルゴリズムが欲しくなります。以下では整数を格納した配列に対して、特定の数字が存在するかどうかを判定して Bool 値で返す問題を考えます。このノートでは

- 線形探索
- ソートを使った探索
- m ブロック法
- 2 分探索法

を紹介します。

2 線形探索法

線形探索法は最も愚直な方法になります。線形探索法では配列の先頭から順番に要素を確認して、探索している要素があったら探索を修了して True で返す、配列を最後まで探索して要素がなければ False で返します。以下に疑似コードを示します。

Algorithm 1 linear search

```
1: data : 与えられた配列
2: x:探索したい数字
3: size : 配列 data の大きさ
4: for  $i = 0; i < size; ++i$  do
5:   if  $data[i] == x$  then
6:     return True
7:   end if
8: end for
9: return False
```

この方法ではもし仮に探している要素が配列内になかったら n 回の比較が必要です。一方で探している要素があれば、そこで比較は終了です。探している要素が配列のどこにあるかわからないですが、平

均的には真ん中付近にあると思うと比較回数は平均しておよそ $n/2$ であることが予想できます。故に線形探索の計算時間は $\mathcal{O}(n)$ であることがわかります。

3 ソートを用いた探索

線形探索では対象とする要素が存在しないことを確認するには配列内の全ての要素を確認する必要があります。これは探索対象の配列が特に規則性もなく並んでいるがために起きます。そのため探索対象となる配列が何らかの規則で並んでいる場合にはより効率の良い探索アルゴリズムが存在することが期待できます。この点で配列を作る段階で、無作為に用意された要素を順番に配列の末尾に追加していくのではなく、ある規則に従って配列を構築する工夫を施すことで、探索に有利となることがわかります。

ここではそのような例としてソートを用いた探索を紹介します。ソートとは配列の要素に大小関係が定義されているときに、大きい順あるいは小さい順に並べることを言います。このノートでは整数が要素となるので整数の大小関係でソートを行います。ソートのアルゴリズムはさまざまなものが知られていますがその詳細は割愛して、要素の値が小さい順にソートされた配列 `data` が与えられているとします。この時、`data[i] < data[i + 1]` なので、もし配列の要素を順番に確認していき、探索したい数字 x が見つからずかつ x より大きな数字となった時点でその要素以後には x が存在しないことがわかるので、探索を打ち切ることができます。

Algorithm 2 linear search with sort

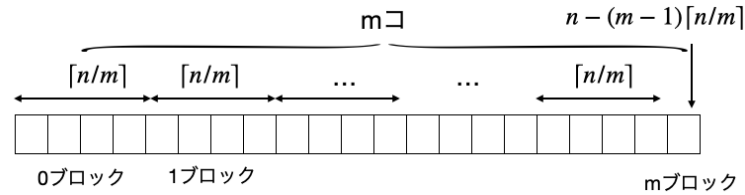
```
1: data : ソートされている与えられた配列
2: x:探索したい数字
3: size : 配列 data の大きさ
4: for  $i = 0; i < size; ++i$  do
5:   if data[i] ≥ x then
6:     break
7:   end if
8: end for
9: if data[i] == x then
10:   return True
11: end if
12: return False
```

このアルゴリズムでは必ずしも配列の全ての要素を比較する必要がないので linear search のアルゴリズムよりも比較回数が少なくて済む可能性があるため linear search アルゴリズムよりも効率的になります。この例から探索アルゴリズムにおいてデータの並びやデータの管理の仕方が重要であることがわかります。

4 m ブロック法

m ブロック法ではソートされた配列全体を m コのブロックに分けて、初めにどのブロックに探索している要素 x があるかを調べてからそのブロック内で探索を行います。ブロックの長さは特に理由がなければできるだけ同じ長さにするのが良いでしょう。そこで与えられた配列全体の長さを n として m コのブロックに分けることを考えます。 n が m で割り切れれば m コの長さ n/m のブロックができます。ま

た割り切れない場合には長さ $\lceil n/m \rceil$ のブロックが $m-1$ コと長さ $n - (m-1)\lceil n/m \rceil$ のブロックが1つとなります。



配列の要素がソートされている時にはそれぞれのブロックの末尾がブロック内の最大要素となります。以下記号を簡単にするために $k = \lceil n/m \rceil$ とします。 $j (= 0, 1, 2, \dots, m-1)$ 番目のブロックの最大要素は $\text{data}[(j+1)k-1]$ で与えられます。そのため先頭のブロックから順番に最大要素と x との大小比較を行い、 $x \leq \text{data}[(j+1)k-1]$ となる最初の j が見つかったら j 番目のブロック内に x が存在する可能性があります。その後に j 番目ブロック内で逐次探索を行います。

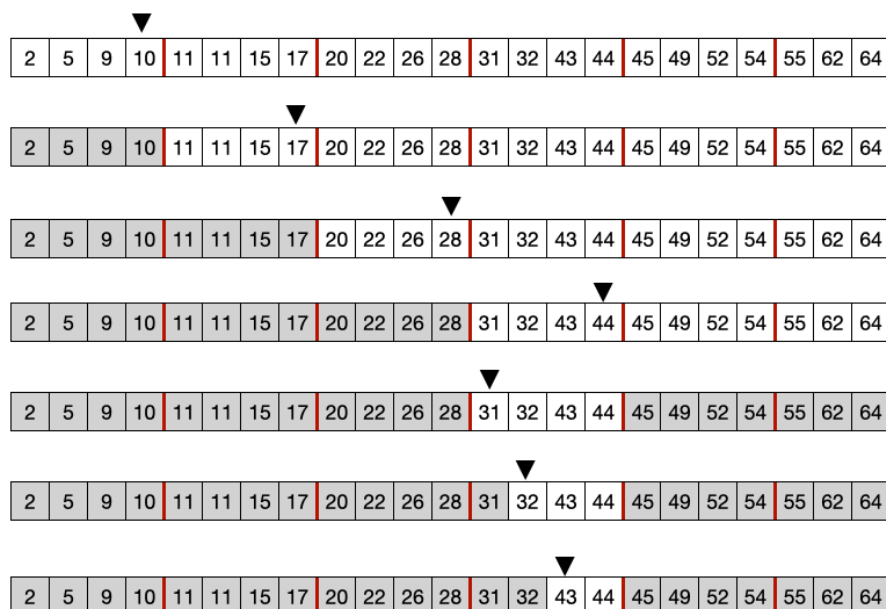
以上のアルゴリズムを具体例を用いてみていきます。対象となる配列は以下で与えられているとします。

2, 5, 9, 10, 11, 11, 15, 17, 20, 22, 26, 28, 31, 32, 43, 44, 45, 49, 52, 54, 55, 62, 64 (1)

この中に $x = 43$ が存在するかどうかを探索します。配列の長さは 23 なので例えば $m = 6$ とします。すると $k = \lceil 23/6 \rceil = 4$ なので最初のブロック $j = 0$ から $j = 4$ までは長さ 4 のブロック、最後のブロックだけ $23 - 4 \times 5 = 3$ の長さとなります。まず初めに、それぞれのブロックの末尾の要素（ブロック内の最大値）と $x = 43$ を比較します。それぞれのブロックの最大値は順に

10, 17, 28, 44, 54, 64 (2)

なので、 $j = 3$ のブロックが候補のブロックとなります。次に $j = 3$ のブロックの先頭 31 から順に要素を見ていくと 43 が3番目に存在するので、探索完了です。もし 40 が探索したい数字だった時にも候補となるブロックは $j = 3$ となります。 $j = 3$ のブロックの先頭から要素を見ていくと 32 の次が 43 となりこの段階で 40 が存在しないことがわかるので、探索完了です。



以上のアルゴリズムを疑似コードで書くと以下ようになります。

Algorithm 3 m-block

```
1: data : ソートされている与えられた配列
2: x:探索したい数字
3: n : 配列 data の大きさ
4:  $k = \lceil n/m \rceil$ 
5:  $j = 0$ 
6: while  $j \leq m - 2$  do
7:   if  $x \leq \text{data}[(j + 1)k - 1]$  then
8:     break
9:   else
10:     $++j$ 
11:   end if
12: end while
13:  $i = jk$ 
14:  $t = \min((j + 1)k - 1, n - 1)$ 
15: while  $i < t$  do
16:   if  $x == \text{data}[i]$  then
17:     return True
18:   else
19:     $++i$ 
20:   end if
21: end while
22: return False
```

このアルゴリズムでの比較回数を見積もります。まず、ブロックの最大値との比較で m 回の比較が必要です。その後に各ブロックでの長さからおおよそ $\mathcal{O}(n/m)$ 回の比較が必要です。ゆえに全体ではおおよそ

$$m + \frac{n}{m} \quad (3)$$

の計算回数となります。ここで m この計算回数が最小となる m は相加相乗平均の不等式から $m = \sqrt{n}$ で与えられ、この時計算回数は $\mathcal{O}(\sqrt{n})$ となります。

5 2分探索法

2分探索でもソートされた配列を対象として探索をします。この方法では探索したい要素 x と探索範囲の中央の要素とを比較して候補となる領域が中央の要素より左にあるか右にあるのかを調べます。左(右)にあることがわかった場合には探索範囲を左(右)の領域に更新します。この操作を繰り返すことで徐々に探索範囲を狭めていきます。変数を導入して、手続きの中での変数の更新の仕方を見ていきます。配列の要素を指す変数 L, R, M を用意します。 L は探索範囲の左側、 R は探索範囲の右側を指します。 M は探索範囲の中央を指します。最初の探索範囲は配列全体なので、初めに $L = 0, R = n - 1$ とします。 $(n$ は配列の長さです。)配列の中央は $M = \lfloor \frac{n-1}{2} \rfloor$ で与えられます。 $n - 1$ が奇数の時には中央の値がないので適当に切り捨てるようにします。探索したい数字 x と $\text{data}[M]$ を比較します。もし $x == \text{data}[M]$ なら $\text{data}[M]$ が探索したい変数なので探索完了です。もし $x < \text{data}[M]$ なら候補となる領域は L $M - 1$

になります。そのため $R = M - 1$ とします。一方でもし $x > \text{data}[M]$ なら候補となる領域は $M + 1$ R となり、そのため $L = M + 1$ として更新します。更新後に $M = \lfloor \frac{L+R}{2} \rfloor$ として同様に探索範囲を狭めていきます。この操作を繰り返していくと探索範囲は1回の更新で半分程度になっていき、探索範囲の長さが1になっても x が見つからなければ、配列内に x は存在しないことがわかり終了となります。

さらに具体的に見るために式 1 で与えられた配列に対して $x = 44$ として2分探索をします。長さは $n = 23$ なので最初は $L = 0, R = 22, M = 11$ となります。 $\text{data}[M] = 28$ で $x = 44$ よりも小さいので $L = M + 1 = 12$ として探索範囲を更新します。更新後に $M = \lfloor \frac{L+R}{2} \rfloor = 17$ となるので $\text{data}[M] = 49$ から $R = M - 1$ として更新します。この操作を繰り返すと $L = 15, R = 16$ に到達します。この次ステップでは $M = \lfloor \frac{L+R}{2} \rfloor = 15$ となりこれは L と一致しますが、そのままこれまでと同じ手続きを繰り返すと $\text{data}[M] = 44$ でこれは探している要素と一致しているので、配列内に要素があることがわかり探索終了です。図 1 に探索中の変数の変化を示します。

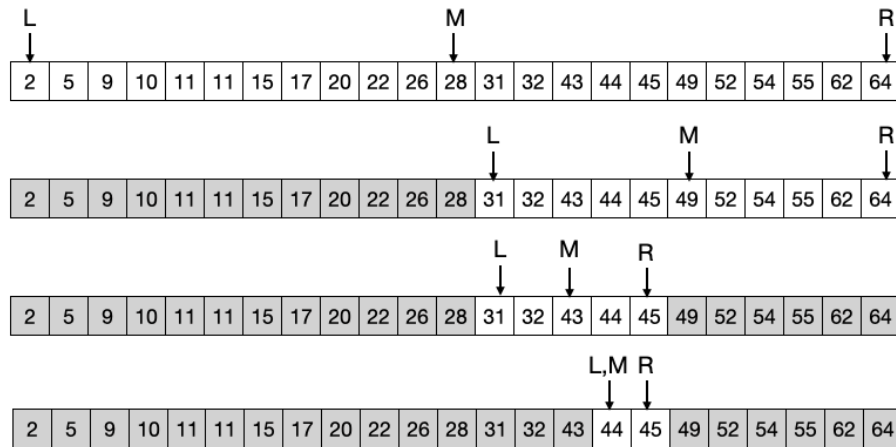


図 1: 式 1 で $x = 44$ としたときの2分探索法の変遷

一方で例えば $x = 45$ の場合を考えます。この場合にも途中までは同じで $L = 15, R = 16$ に到達します。次の操作で $M = \lfloor \frac{L+R}{2} \rfloor = 15$ で $\text{data}[M] = 44$ です。これは $x = 45$ より小さいので $L = M + 1 = 16$ とすると $L = R$ となります。 $L = R$ になったら繰返しの手続きを終えて $\text{data}[L]$ と x を比較します。今回は $\text{data}[L] = x = 45$ なので配列内に要素が見つかったとして終了となります。一方で配列内に探索対象の要素がない場合には $L = R$ で繰返しの手続きを終えても $\text{data}[L]$ と x は一致しません。その場合には配列内に要素はないとして探索終了となります。

以上の手続きを疑似コードで書くと以下ようになります。

Algorithm 4 binary search

```
1: data : ソートされている与えられた配列
2: x:探索したい数字
3: n : 配列 data の大きさ
4: if  $x < \text{data}[0]$  または  $\text{data}[n-1] < x$  then
5:   return False
6: end if
7:  $L = 0$ 
8:  $R = n - 1$ 
9: while  $L < R$  do
10:   $M = \lfloor \frac{L+R}{2} \rfloor$ 
11:  if  $x == \text{data}[M]$  then
12:    return True
13:  else
14:    if  $x < \text{data}[M]$  then
15:       $R = M - 1$ 
16:    else
17:       $L = M + 1$ 
18:    end if
19:  end if
20: end while
21: return  $\text{data}[L] == x$ 
```

2分探索では探索範囲の長さが半分に減っていくので典型的に $O(\log(n))$ の計算量で探索が終了します。

6 2分探索木を用いた2分探索法

先に紹介した2分探索法ではデータはソートされた配列として与えられました。探索の計算量は $O(\log n)$ となりますが、一般に与えられた配列をソートするには $O(n \log n)$ 程度の計算が必要です。また多くの問題でデータは一定であることは少なくソートされた配列にデータを追加するには $O(n)$ の計算量が必要になります。そこで配列ではないデータ構造として木構造を用いて2分探索を行う方法を考えてみます。

6.1 木構造の復習

データ構造における木構造は、節点と枝から構成されています。接点はデータを蓄え、枝は接点同士を繋ぎます。木構造では枝は接点のある関係で結び、枝の長さには意味はありません。木構造には特別な接点として根が一つあり根から接点が連なっていきます。通常木構造を描画する時には根を一番上に描き順番に下に伸びていくように描きます。木構造では全ての接点は枝を介して順番に何回か接点を辿っていくと根に到達します。この根に至るまでの最短の経路は一つになります。枝で繋がった二つの接点においてより根に近い方の接点を親、遠い方を子と表現します。根には親は存在せず、子のみを持つことができます。一方で子を持たない接点を葉と呼びます。ある接点から根までに至る枝の数をその接点の深さと呼び、特に根から最も深い接点の葉までの枝の数を木の高さと呼びます。

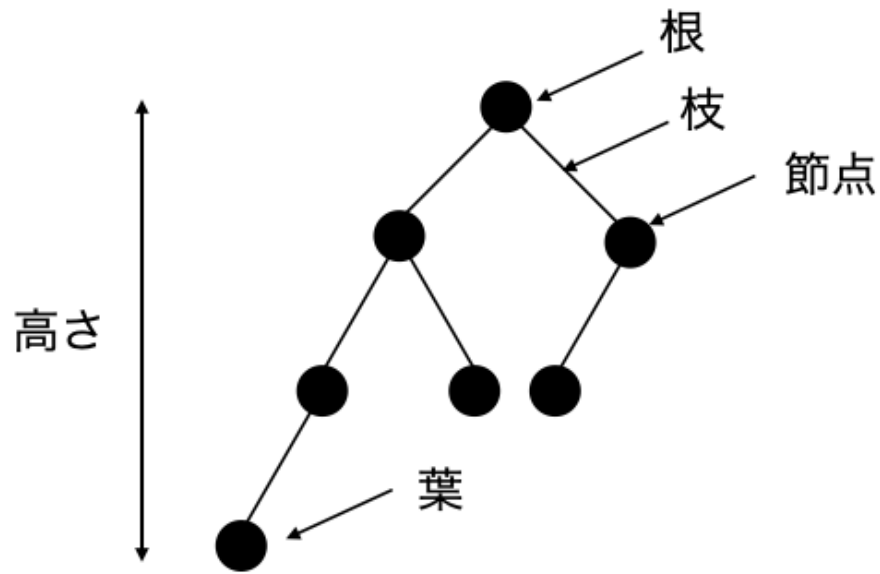


図 2: 木構造の例

一般の木構造では接点に幾つの子があっても良いが高々2つの子しか持たない木構造を二分木と呼びます。根から葉までの距離が全ての葉で等しい時には完全2分木と呼びます。完全二分木は接点がバランスよく配置されている木構造になります。この時、木の高さを h とすると接点の数は $2^{h+1} - 1$ で与えられます。接点の数はデータの数に対応していますから、ちょうど $2^{h+1} - 1$ コのデータ数が存在しないこともあり得ます。その場合には先の完全2分木の一番左の葉から順に接点を追加していくことでバランスを保つことができます。そのようにして作られる二分木も完全2分木と呼ばれることがあります。

6.2 2分探索木

2分木を用いた2分探索法のために2分探索木を考えます。2分探索木では任意の子を持つ接点に対して左の子の値は接点の値より小さく、接点の値は右の子の値より小さいような2分木です。具体例を見てみます。

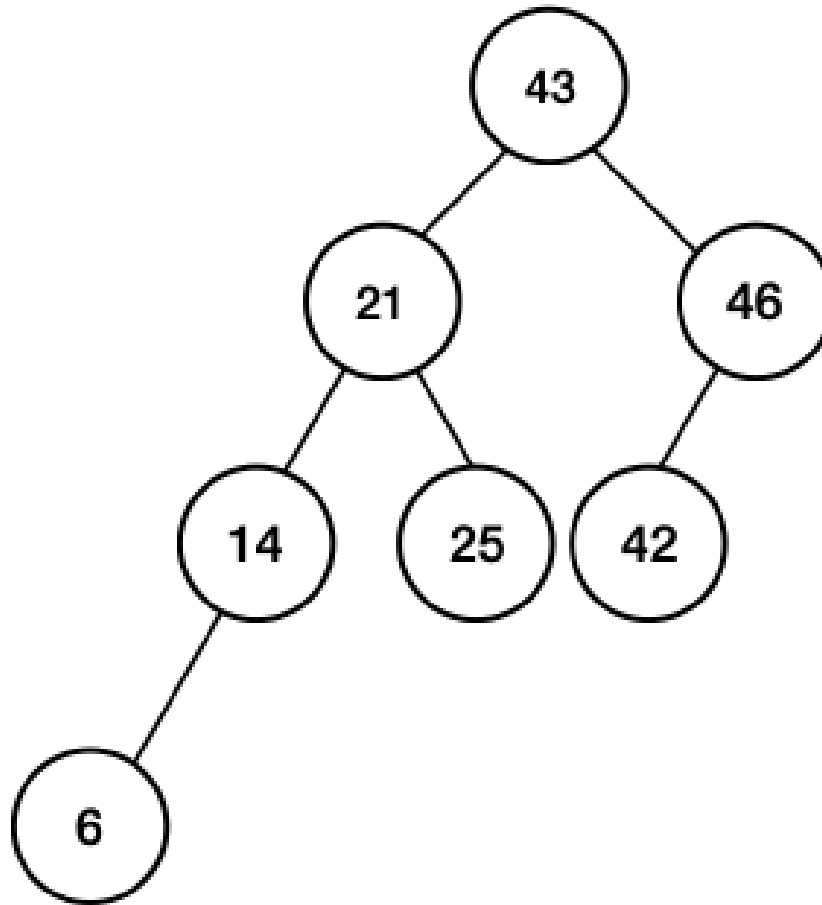


図 3: 二分探索木の例

この例では根の左の子 (21) は根の値 (43) より小さく、根の値は右の子 (46) より小さくなっています。他の接点についても同様のことが言えます。

では次に 2 分探索木におけるデータの探索と新しいデータを追加する方法を考えます。今、図 3 で与えられる二分探索木において $x = 25$ を探索したいとします。データの探索では根から順番に数字を見ていきます。根の値は 43 なので $x = 25$ があるとすれば根の左側にあるはずですが、そこで次に左の子に移動します。そこでの値は 21 で $x = 25$ とは異なります。 x の方が大きいので右の子に移動します。するとそこで接点の値が 25 でこちらは探索している要素となります。

一方で $x = 27$ を探索することを考えます。この場合にも先と同様の考え方で 25 を値にもつ節点まで移動します。その後接点と $x = 27$ の値が異なり、 x の方が大きいので左の子を見ますが、この接点は左の子がありません。ゆえにデータには $x = 27$ が存在しないことが結論づけれます。実はデータの追加も同じ考え方で行うことができます。2 分探索木におけるデータの追加では適当な箇所にデータを追加することはできず、親と 2 つの子の関係を尊重しなくてはなりません。実は数字の追加はデータの探索で最後に探索した接点に追加をすることで、実現できます。数字 27 を追加したいと思うと、先と同様にして 25 が蓄えられた接点まで移動できます。探索の場合にはその接点と探索したい数字を比較して値が存在するかを確認していました。データの追加でも追加したい値がすでにあればそれ以上何もしなくて良いですが、追加したいデータが存在しないことがわかったら最後に探索した場所が、追加したいデータがあるべき場所のはずです。数字 27 は 25 より大きいので 25 の接点の右の子として保存すべきであることがわかります。

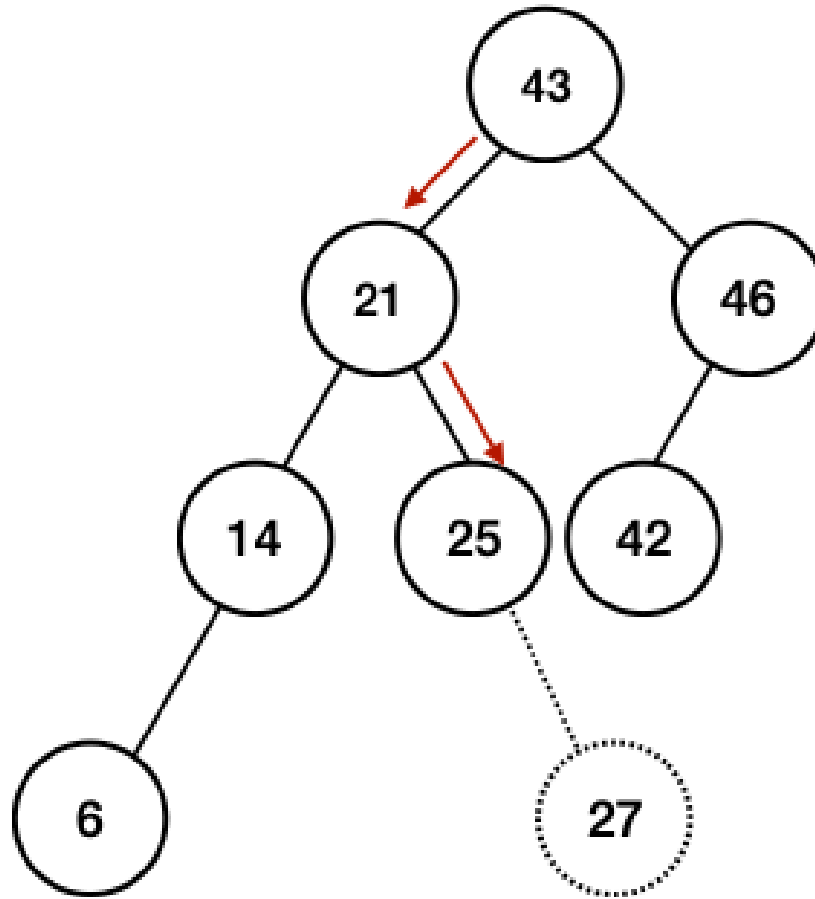


図 4: 二分探索木における探索とデータの追加

このようにして順番にデータを追加していくことで2分探索木を構成することができます。

6.3 疑似コードによる実装

それでは実際に疑似コードを用いて以上の2分探索木とそれによる2分探索のアルゴリズムを実装してみます。まず2分探索木の実現が必要です。2分探索木では各節点における値と二つの子を指すデータが必要になります。ここでは各節点を Node という構造体で表現します。

メンバ変数	説明
value_	節点の値
right_	右の子節点へのポインタ
left_	左の子節点へのポインタ

メソッド	説明
node (int v)	コンストラクタ (v を value に格納する。right, left は none に初期化する。)

図 5: Node のメンバ変数

対象とする節点が2分探索木の葉に相当する場合には子がないのでその場合には right と left は none

になります。二分探索木はこの構造体 `Node` の配列 `Nodes` を含む `BinarySearchTree` 構造体で表現できます。`BinarySearchTree` 構造体は以下のメンバ変数とメソッドを持ちます。

メンバ変数	説明
<code>Nodes_</code>	<code>Node</code> の配列
メソッド	説明
<code>BinarySearchTree()</code>	コンストラクタ、 <code>Nodes</code> を初期化。
<code>void add_node(int v)</code>	節点を追加する
<code>[node, str] get_parent(int v)</code>	親となる節点を探す
<code>bool has_value(int v)</code>	値があるなら <code>True</code> ないなら <code>False</code> を返す

図 6: `BinarySearchTree` のメンバ変数とメソッド

`Nodes` は `Node` の配列で二分探索木のデータ構造を保持します。`add_node` メソッドは整数 v を引数に持ち v を二分探索木ないの適当な位置に追加します。メソッドのアルゴリズムは以下のようになります。

Algorithm 5 `BinarySearchTree : add_node`

```

1:  $v$  : 引数、整数型
2: node_temp = node(v)
3: if Nodes_ が空でない then
4:   p, dir = get_parent(v) !親となる節点と右か左か
5:   if dir == "" then
6:     return
7:   end if
8:   if dir == "left" then
9:     p.left = node_temp
10:  else
11:    p.right = node_temp
12:  end if
13: end if
14: Nodes_ に node_temp を追加
15: return

```

四行目で後述する `get_parent` を用いて、親となる節点とその右の子に追加すべきか左の子に追加すべきかを調べています。`Nodes_` はコンストラクタで初期化されているので、最初に追加する時には 3 行目の条件に当てはまらずそのまま 14 行目で `Nodes_` に追加されます。その後は `dir` が空でない限りは新たな節点が `left` か `right` に追加されます。一方ですでにある節点が新しく追加する節点の子になることはないので `Nodes_[0]` は必ず根になります。`get_parent` のアルゴリズムは以下のようになります。

Algorithm 6 BinarySearchTree : get_parent

```
1: v : 引数、整数型
2: node = Nodes_[0]
3: while node が空でない do
4:   p = node
5:   if p.value_ == v then
6:     return [ none, "" ]
7:   end if
8:   if v < p.value_ then
9:     dir = "left"
10:    node = p.left_
11:  else
12:    dir = "right"
13:    node = p.right_
14:  end if
15: end while
16: return [p,dir]
```

add_node の構成から Nodes_[0] は二分木の根なので、get_parent では根から順番に節点を探索していきます。メソッドの return は親となる節点 p とその接点の左の子になるのか右の子になるのかの dir を返します。今考えている二分探索木では節点の値よりも、その節点の左の子の方が値が小さく、右の子の方が値が大きくなります。そのため get_parent メソッドでは根から順番に節点の値と v を比較して、v の値を持つ節点のあるべき場所（親となるべき節点）を探します。探していく中で、もし v と同じ値を持つ節点があれば v を追加する必要はないのでその時点で return します。この時には返り値は空となります（6 行目）。

has_value メソッドは現在の二分探索木に引数で与えられる値 v がすでにある場合には True、ない場合には False を返します。こちらは先の get_parent メソッドを用いて実現することができます。

Algorithm 7 BinarySearchTree : has_value

```
1: v : 引数、整数型
2: p,dir = get_patent(v)
3: if dir == "" then
4:   return True
5: end if
6: return False
```

以上の BinarySearchTree クラスを用いて二分探索木を用いた二分探索法を実現することができます。ここではデータの追加も込みで BinaryTreeSearch クラスを定義します。BinaryTreeSearch メソッドは以下のメンバ変数とメソッドを持ちます。

メンバ変数	説明
btree_	BinarySerchTree クラスのインスタンス
メソッド	説明
BinarySearchTree()	コンストラクタ、btree_を初期化。
void append(int v)	節点を追加する
bool find(int v)	値 v が存在するかどうか二分探索する

append メソッドは BinarySearchTree の add_node を用いてデータを追加します。

Algorithm 8 BinaryTreeSearch : append

- 1: v : 引数、整数型
 - 2: btree_.add_node(v)
-

find メソッドは BinarySearchTree の has_value を用いることができます。

Algorithm 9 BinaryTreeSearch : find

- 1: v : 引数、整数型
 - 2: btree_.has_value(v)
-

以上のアルゴリズムを用いて具体的にどのような二分探索木が構成されるかを確認してみます。入力として

$$6, 9, -1, -4, 7, 10, 0 \quad (4)$$

という数字を順番に二分探索木に BinaryTreeSearch:append() のメソッドを用いて追加すると以下のようになります。

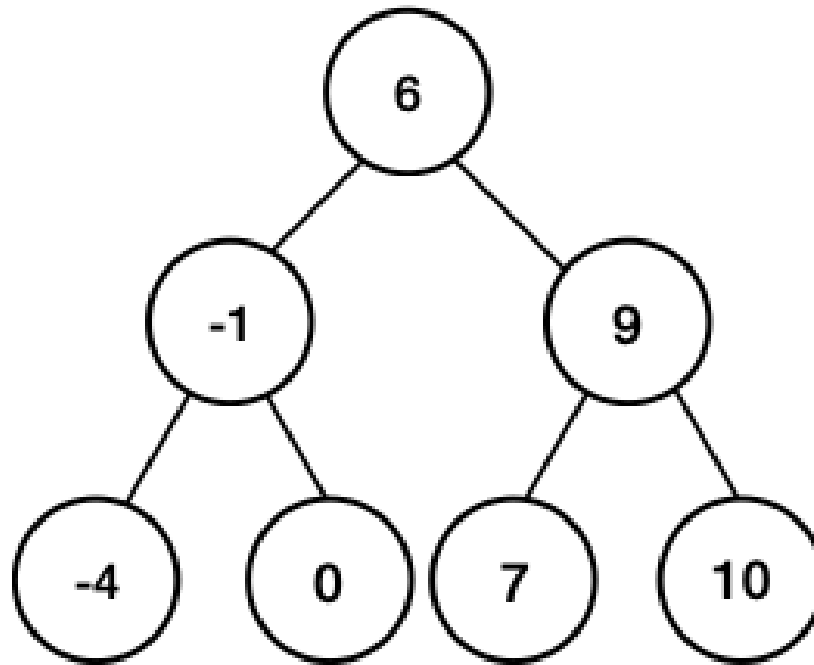


図 7: 二分探索木の例

一方で同じ配列を以下のようにソートした後に `BinaryTreeSearch:append()` のメソッドで追加します。

`- 4, -1, 0, 6, 7, 9, 10` (5)

この時に生成される二分探索木は

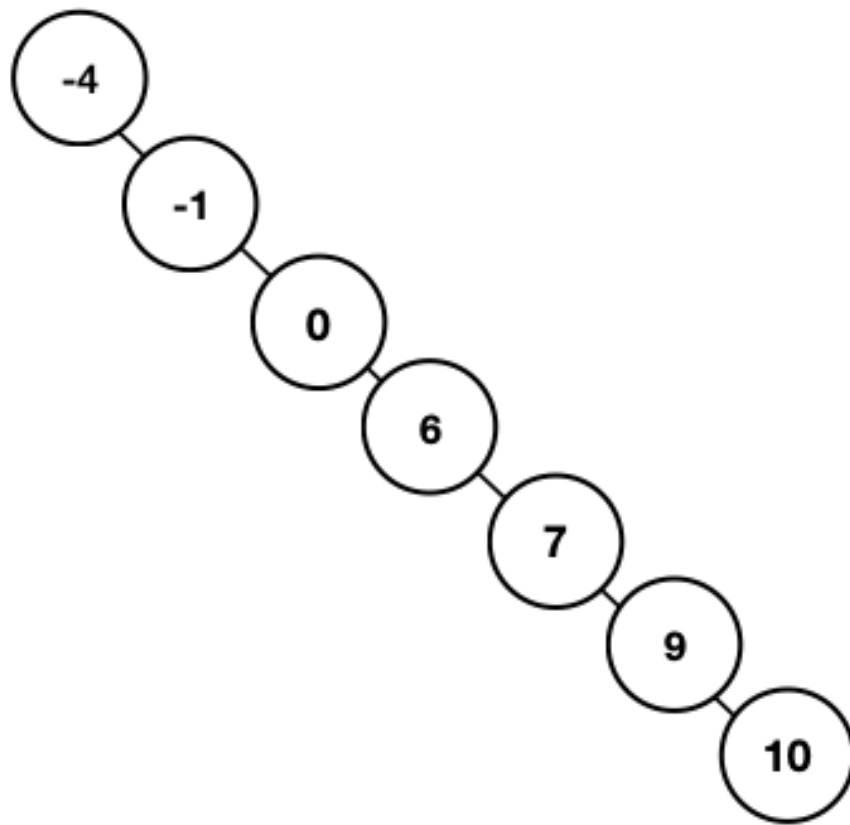


図 8: 二分探索木の例

このように同じ配列でも二分探索木へのデータの追加の順番を変えると最終的に生成される二分探索木が異なります。図 7 の場合には木の左右のバランスがよく数字の探索では高々3 回程度の数字の比較で終わります。一方で図 8 では最悪 7 回程度の比較が必要になります。このように二分探索木は同じデータでも追加する順番によって木の構造が異なり、木の左右のバランスが取れていないような木も生成される可能性があります。その場合には、数字の探索は最悪の場合要素の数分だけの比較が必要なり、非効率になってしまいます。

7 ハッシュ探索

ハッシュ探索ではデータをハッシュ関数と呼ばれる関数を用いて配列 `data` に格納します。以下では配列 `data` の大きさを m として、格納するデータとしては整数を考えます。ハッシュ関数 $\text{hash}(x)$ はデータ x に対して 0 から $m-1$ までのどれかの数字を返す関数です。ハッシュ関数は同じ引数に対しては常に同じ値を返します。一方で、異なる引数に対して必ずしも異なる値を返すわけではありません。今ハッシュ関数として引数である整数 x を m で割ったあまり j を返すハッシュ関数を採用します。 j は 0 から $m-1$ まで整数です。ハッシュ探索では配列の j 番目の位置 `data[j]` にデータを格納します。このハッシュ関数の定義では、異なる x でも同じハッシュ値が得られることがあります。この場合の対応はいくつか流儀があるようですが、ここで `data[j]` を配列として持っておいて順番にデータを格納をします。

データの探索をする時には、探索したいデータ x のハッシュ値 j を計算し、`data[j]` の配列を順番に探すことで x が存在するかどうかわかります。以下ではハッシュ探索を `HashSearch` というクラスとして実装してみます。

	メンバ変数	説明
	data_	データを格納する配列
	size_	配列 data_ の大きさ
メソッド	説明	
HashSearch(int m)	コンストラクタ, size_ を引数 m で初期化。data_ をサイズ m の配列として初期化。	
get_hash(int x)	ハッシュ関数	
search(int x)	データを探索	
append(int x)	データを追加	

図 9: HashSearch のメンバ変数とメソッド

HashSearch クラスのメソッドの疑似コードは以下の通りとなります。

Algorithm 10 hash search : get_hash(int x)

```

1: x : 引数 (整数)
2: return x % size_

```

Algorithm 11 hash search : search(int x)

```

1: x : 引数 (整数)
2: j = get_hash(x)
3: for i=0; i<data_[j].size(); ++ i do
4:   if data_[j][i] == x then
5:     return True
6:   end if
7: end for
8: return False

```

Algorithm 12 hash search : append(int x)

```

1: x : 引数 (整数)
2: j = get_hash(x)
3: data_[j].append(x)

```

参考文献

[1] 辻真吾、下平英寿、「Python で学ぶアルゴリズムとデータ構造」